Pashov Audit Group

Polygun Security Review

February 17th 2026 - February 26th 2026

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Polygun

Polygun is a copy trading platform built on top of Polymarket that enables users to automatically mirror trades from key opinion leaders. It provides wallet management, limit orders, fee processing, referral tracking, and commission distribution for prediction market trading.

6. Executive Summary

A time-boxed security review of the **toantt208/polygun_audit** repository was done by Pashov Audit Group, during which **0xAlix2**, **Shurikenzer**, **defsec**, **0xaudron** engaged to review **Polygun**. A total of **53** issues were uncovered.

Protocol Summary	
Project Name	Polygun
Protocol Type	Polymarket copy trading
Timeline	February 17th 2026 - February 26th 2026

Review commit hash:
- 360d6fe452bd855b4f3fe10e4d555a1372d436e6 (toantt208/polygun_audit)

Fixes review commit hash:
- 42f1f1e8b72623b7e22098c9d445af7058fdf009 (toantt208/polygun_audit)

Scope

`commissions.ts` `copy-trading.ts` `deploy-safe.ts` `fee-transfers.ts` `index.ts` `kol-promo.ts` `limit-orders.ts` `market-cache.ts` `referrals.ts` `schema.ts` `temporary-wallets.ts` `users.ts` `wallets.ts`

7. Findings

Findings count

Severity	Amount
High	2
Medium	10
Low	41
Total findings	**53**

Summary of findings

ID	Title	Severity	Status
[H-01]	Race condition in limit order processor allows fee bypass	High	Resolved
[H-02]	Promo claim race condition allows double payment	High	Resolved
[M-01]	Referral withdrawal double claim via TOCTOU race	Medium	Resolved
[M-02]	Non-atomic pause allows active trades during pause	Medium	Resolved
[M-03]	Mute/unmute notifications ignore intent service only toggle	Medium	Resolved
[M-04]	Missing slot check in `createUserPromo` allows budget overrun	Medium	Resolved
[M-05]	Failure to verify USDC transfer results can lead to loss of funds	Medium	Resolved
[M-06]	`mapHistoryRow` unbound `this` in map() causes runtime TypeError	Medium	Resolved
[M-07]	Non-atomic commission balance operation enables race conditions	Medium	Resolved
[M-08]	Promo claim updates state without validation	Medium	Resolved
[M-09]	Unencrypted database communication	Medium	Resolved
[M-10]	Migration fails due to non-existent serial type	Medium	Resolved
[L-01]	pauseSubscription() overwrites pause reason without check	Low	Resolved

ID	Title	Severity	Status
[L-02]	`reactivateFollow()` does not verify the follow is paused	Low	Resolved
[L-03]	No application path to set user ban `isBanned`	Low	Acknowledged
[L-04]	Commission creation default status vs schema	Low	Resolved
[L-05]	Wallet mapping omits `deletedAt`	Low	Resolved
[L-06]	User mapping omits copy trading field	Low	Resolved
[L-07]	Soft delete does not update `updatedAt`	Low	Resolved
[L-08]	`telegramId` not unique in the wallets table	Low	Resolved
[L-09]	`referralCode` not unique in the users table	Low	Resolved
[L-10]	Potential rounding error in wei-to-token conversion	Low	Resolved
[L-11]	Copy trading actions lack authorization check	Low	Resolved
[L-12]	Pending commission can be restored after successful USDC transfer	Low	Resolved
[L-13]	Missing `feePercent` in fee transfer job data	Low	Resolved
[L-14]	Incorrect price calculation in limit order processor	Low	Resolved
[L-15]	Limit order may be marked fully filled due to accounting mismatch	Low	Resolved
[L-16]	Multiple unbounded queries risk OOM during backlog processing	Low	Resolved
[L-17]	Database client has no statement timeouts	Low	Resolved
[L-18]	`incrementCompletedTrades` swallows errors	Low	Resolved
[L-19]	No check constraints on 12+ status enum text columns	Low	Resolved
[L-20]	`fee_transfers` table has no deduplication on `tradeId`	Low	Resolved
[L-21]	FindOrCreateByTelegramId prevents reregistration of soft-deleted users	Low	Resolved
[L-22]	Timestamp columns use mixed timezone handling	Low	Resolved
[L-23]	`setReferrer` allows changing referrer after initial set	Low	Resolved

ID	Title	Severity	Status
[L-24]	Missing input validation at database service layer	Low	Resolved
[L-25]	HasPromoRestriction fails open on database error	Low	Resolved
[L-26]	Missing unique constraint allows duplicate promos	Low	Resolved
[L-27]	`Math.random()` used for share codes and referral codes	Low	Resolved
[L-28]	`updateFollowSettings` does not check `deletedAt`	Low	Resolved
[L-29]	Unsafe usage of `sql.raw()` for `INTERVAL` in getTotalFees	Low	Resolved
[L-30]	Schema enforces uniqueness only via application-level checks	Low	Resolved
[L-31]	Inconsistent error handling at `findByTokenId`	Low	Resolved
[L-32]	`UserService.withRetry()` implements linear not exponential backoff	Low	Resolved
[L-33]	Kol-promo.ts checkAndUnlockPromo missing status guard on `UPDATE`	Low	Resolved
[L-34]	Auto claim cron includes banned users	Low	Resolved
[L-35]	PostgreSQL rounds decimal(18, 6) inserts causing potential overcharges	Low	Resolved
[L-36]	`uniqueFollow` index is not a unique constraint	Low	Resolved
[L-37]	Self copy trading allows users to amplify own trades into infinite loop	Low	Resolved
[L-38]	ReferralService.mapRowToUser omits `isBanned` field	Low	Resolved
[L-39]	Commission permanently lost if Redis enqueue fail	Low	Resolved
[L-40]	checkDailyLimit() type mismatch bypasses daily limit	Low	Resolved
[L-41]	`claimPromo` accepts arbitrary amounts	Low	Resolved

High findings

[H-01] Race condition in limit order processor allows fee bypass

Severity

Impact: High

Likelihood: Medium

Description

Users are not charged a fee when their limit order is fully filled due to a race condition between two concurrent jobs processing the same limit order.

More specifically, the limit order processor worker (`packages/queue/src/workers/limit-order-processor.worker.ts`) processes `OrderFilled` events for limit orders. Each event is submitted as a job, and up to 5 jobs can be processed concurrently. The processor retrieves the limit order from the database and checks whether the order is fully filled based on the current filled amount (`currentFilled`) and the new fill amount (`fillAmount`). If the order is considered fully filled, the fee is transferred from the user wallet to the master wallet. Otherwise, no fee is charged.

```
// Step 1: Look up order in database by orderHash
const limitOrder = await limitOrderService.findByOrderHash(orderHash);
...
// Step 5: Determine if fully filled (99% threshold)
const originalSize = parseFloat(limitOrder.originalSize || '0');
const currentFilled = parseFloat(limitOrder.filledSize || '0');
const newFilledSize = currentFilled + fillAmount;
const isFullyFilled = originalSize > 0 && newFilledSize >= originalSize * 0.99;

// Step 6: Update fill tracking in database
try {
    await limitOrderService.updateFill(limitOrder.id, fillAmount, isFullyFilled);
} catch (error) {
    ...
}
...
```

However, when two jobs for the same limit order are processed concurrently, `newFilledSize` can be calculated incorrectly because `currentFilled` is outdated. As a result, `newFilledSize` may be smaller than the actual on-chain filled amount, causing `isFullyFilled` to be evaluated as `false` , even though the order is already fully filled on-chain.

Example scenario:

1. A limit order is created with `originalSize = 200`.
2. Two users fill the order with amounts of `130` and `70`.
3. Two jobs are processed concurrently:
4. Job 1 processes `fillAmount = 130`
5. Job 2 processes `fillAmount = 70`
6. Both jobs read `currentFilled = 0` from the database.
7. Job 1 calculates `newFilledSize = 130`
8. Job 2 calculates `newFilledSize = 70`
9. Neither job detects the order as fully filled (`200`), and therefore the fee is not charged.

As a result, the order becomes fully filled on-chain, but the system fails to recognize it as fully filled off-chain.

Impact

If this scenario occurs, the limit order may never be marked as fully filled off-chain due to incorrect fill tracking. Consequently, the fee transfer logic is not triggered, resulting in a loss of protocol revenue.

Recommendations

Modify the logic so that the fill update occurs before determining whether the order is fully filled.

Specifically:

- `LimitOrderService::updateFill()` should update `filledSize` atomically and in isolation (e.g., using a database transaction).
- The updated `filledSize` returned from `updateFill()` should then be used to determine whether the order is fully filled.
- The fully filled check should be based on the latest persisted state, not on a previously read value.

[H-02] Promo claim race condition allows double payment

Severity

Impact: High

Likelihood: Medium

Description

The **promo claim handler** processes a user's button press by reading the promo status from the database and, if `status === 'pending'`, enqueuing a BullMQ job that executes the on-chain USDC transfer. Two concurrent Telegram webhook callbacks from duplicate deliveries observe `status === 'pending'`, and each enqueues an independent job as jobId since BullMQ generates a random UUID per call when no jobId is provided.

```
// packages/telegram-ui-v2/src/handlers/promo.ts
if (!userPromoDetails || userPromoDetails.userPromo.status !== 'pending') return;

await promoClaimQueue.add('claim-promo', {
    userPromoId: userPromoDetails.userPromo.id,
    amount: ...,
    //@audit-info no jobId: BullMQ assigns a random UUID per call as jobId is not given
});
```

Both workers then execute `transferUsdc()` before `claimPromo()` WHERE `status = 'pending'` guard runs. The guard stops the second database write but does not stop the second on-chain transfer, which is already confirmed.

Impact: User receives double the promo amount. The second transfer is unrecorded in the database and irrecoverable through application-layer tooling.

Note: This finding stems from `packages/telegram-ui-v2/src/handlers/promo.ts`; however, the function `claimPromo()` is in scope (`packages/database/src/services/kol-promo.ts`), and this is a higher layer for the claim promo function.

Recommendations

- Execute `UPDATE user_promos SET status='processing' WHERE id=? AND status='pending'` in the handler before `queue.add()`. If zero rows are affected, return without enqueueing. This is the primary fix.
- Deterministic `jobId`: Pass `{ jobId: promo-claim-${userPromoId}}` to `promoClaimQueue.add()`. BullMQ will reject duplicate jobs for the same ID already in waiting or active state.
- Guard before transfer in the worker. Reorder so `claimPromo()` executes first and `transferUsdc()` only fires after exclusive DB ownership is confirmed.

Medium findings

[M-01] Referral withdrawal double claim via TOCTOU race

Severity

Impact: High

Likelihood: Low

Description

The `referral_withdraw` callback handler reads the user's `pendingCommission`, checks it meets the minimum, resets it to 0, then enqueues a withdrawal job with the read amount. The read and the reset are **not atomic**: two rapid clicks can both read the same non-zero balance before either resets it, resulting in two jobs that each pay out the full amount from the master wallet.

Three issues combine to make this exploitable:

1. **TOCTOU gap:** The balance is read (`findOrCreateByTelegramId`) and reset (`resetPendingCommission`). These are separate queries with no lock or transaction. A second request can read the same balance before the first request resets it.
2. **Non-deterministic job ID:** The job ID is `referral-withdraw-${user.id}-${Date.now()}` . Because `Date.now()` differs between requests, BullMQ treats them as distinct jobs and accepts both. A deterministic ID (e.g. `referral-withdraw-${user.id}`) would cause BullMQ to reject the duplicate.

```
bot.action('referral_withdraw', async (ctx) => {
    // ...
    const user = await userService.findOrCreateByTelegramId(...);       // READ
    const pendingCommission = parseFloat(user.pendingCommission || '0');
    if (pendingCommission < REFERRAL_MIN_WITHDRAWAL) { ... }            // CHECK

    await userService.resetPendingCommission(user.id);                 // RESET (not atomic
with read)

    await referralWithdrawQueue.add('referral-withdraw', {
      amount: pendingCommission,                                        // stale value from
READ
      // ...
    }, {
```

```
      jobId: `referral-withdraw-${user.id}-${Date.now()}`,              // non-deterministic
    });
});
```

1. **Worker pays blindly:** The worker (`referral-withdraw.worker.ts`) trusts the `amount` in the job data and transfers it from the master wallet without re-checking the user's current balance or pending commission. It also runs with `concurrency: 5` , so both jobs can execute in parallel.

```
const { amount, destinationAddress } = job.data;  // trusts amount from job
const amountInUnits = BigInt(Math.floor(amount * 1e6));
const result = await sdk.transferUsdc({ to: destinationAddress, amount: amountInUnits });
```

Race timeline:

```
Click 1                              Click 2
_______                              _______

L1866: read user
       pendingCommission = 10
                                     L1866: read user
                                            pendingCommission = 10
L1871: check >= $5 → passes
                                     L1871: check >= $5 → passes
L1883: reset to 0
                                     L1883: reset to 0 (no-op)
L1889: enqueue job, amount: 10
                                     L1889: enqueue job, amount: 10
       (different jobId via Date.now)
```

Both jobs run. Master wallet pays out $10 twice. User had $10 in pending commission.

Recommendations

Consider using a **deterministic job ID** and move the balance reset to the worker:

1. Change the job ID to `referral-withdraw-${user.id}` (drop `Date.now()`). BullMQ rejects a second job while one with the same ID is still in the queue; instant deduplication, no race condition.
2. Remove `resetPendingCommission` from the handler. Instead, have the **worker** read the current `pendingCommission` , reset it, and transfer. Since BullMQ guarantees only one job per ID runs at a time, there is no concurrent access.
3. Set `removeOnComplete: true` and `removeOnFail: true` on the job options so the ID is freed after the job finishes, allowing the user to withdraw again later.

[M-02] Non-atomic pause allows active trades during pause

Severity

Impact: High

Likelihood: Low

Description

`pauseAllUserSubscriptions` performs two separate updates without a transaction. If the first succeeds and the second fails, the user is marked as paused, but their following rows stay active. Copy-trade execution does not filter on user-level pause, so trades continue for that user.

`pauseAllUserSubscriptions`:

1. Update **users**: set `copyTradePausedAt`, `copyTradePausedReason`, `updatedAt` for the user.
2. Update **copy_trading_follows**: set `isActive = false`, `autoStoppedAt`, `autoStoppedReason` for all of that user's active, non-deleted follows.

If (1) commits and (2) fails (e.g., DB error, timeout):

- The user row has `copyTradePausedAt` set → UI and `isUserCopyTradingPaused()` treat the user as paused.
- Follow rows are unchanged → still `isActive = true`.

The code that builds the list of followers for copy execution (`getActiveFollowersByLeaderAddress`, `getActiveFollowersWithSafeStatus`) filters only on `copy_trading_follows.is_active = true` and `deleted_at IS NULL`. It does not filter on `users.copy_trade_paused_at`. So this user's follows are still returned, and copy-trade jobs are still enqueued and executed.

This results in the user believing they are paused (and the user table states so), but copy trades continue. They can receive trades and lose funds or take risks they thought were paused.

Recommendations

Consider running both updates inside a single database transaction (e.g., `this.db.transaction(async (tx) => { ... })` using `tx` for both updates). That way, either both commit, or both roll back, and the user is never left in a "paused at user level but follows still active" state.

[M-03] Mute/unmute notifications ignore intent service only toggle

Severity

Impact: Low

Likelihood: High

Description

The handler passes two arguments to set the desired mute state; the database service accepts only one and always toggles. The second argument is ignored, so "Mute" and "Unmute" both behave as a single toggle and the intended behavior never works.

Handler (`packages/telegram-ui-v2/src/handlers/copy-trading.ts`): Two actions call the service with `(followId, true)` or `(followId, false)` :

```
// Mute Notifications
await copyTradingService.toggleNotificationsMuted(followId, true);

// Unmute Notifications
await copyTradingService.toggleNotificationsMuted(followId, false);
```

Service (`packages/database/src/services/copy-trading.ts`): Signature has one parameter; implementation always flips the current value:

```
async toggleNotificationsMuted(followId: number): Promise<boolean> {
    const follow = await this.getFollowById(followId);
    // ...
    const newMutedStatus = !follow.notificationsMuted;
    await this.db.update(copyTradingFollows)
        .set({ notificationsMuted: newMutedStatus, ... })
        .where(eq(copyTradingFollows.id, followId));
    return newMutedStatus;
}
```

So when the user taps "Mute", the service toggles (if already muted, they get unmuted). When they tap "Unmute", the service toggles again. The UI suggests "set to muted" / "set to unmuted", but both actions are the same toggle.

Users cannot reliably set a specific state from the two buttons.

Recommendations

Consider changing the service to accept the desired state and set it: e.g. `setNotificationsMuted(followId: number, muted: boolean): Promise<void>` , and in the implementation set `notificationsMuted: muted` instead of toggling.

[M-04] Missing slot check in `createUserPromo` allows budget overrun

Severity

Impact: High

Likelihood: Low

Description

`findActivePromoByRefCode` checks for available slots before returning a promo:

```
const result = await this.db.query.kolPromos.findFirst({
    where: (kp, { eq, and, lt }) =>
        and(
            eq(kp.refCode, refCode),
            eq(kp.isActive, true),
            lt(kp.usedSlots, kp.availableSlots)
        ),
});
```

`createUserPromo` does not:

```
await tx.update(kolPromos)
    .set({
        usedSlots: sql`${kolPromos.usedSlots} + 1`,
        updatedAt: new Date(),
    })
    .where(eq(kolPromos.id, kolPromoId));
```

It only does `used_slots = used_slots + 1` and inserts into `user_promos`, with no condition that a slot is still available.

So more users than `availableSlots` can get a user_promo and claim the full amount, and total payouts can exceed the intended cap.

Promos are tied to a ref code. When a user opens a KOL link (`/start ref_<code>`), the bot calls `findActivePromoByRefCode` and, if there is room, stores the promo in session as `pendingKolPromo`. No slot is reserved. Only when the user finishes wallet creation does the app call `createUserPromo`.

Because of that time gap, multiple users can pass the check with the same last slot(s), then all complete wallet creation, and each gets a user_promo. Concurrent calls to `createUserPromo` can also both increment and insert.

So slots are over-allocated, and the budget has exceeded.

Recommendations

Consider checking for available slots in `createUserPromo`, so that the promo cannot be created if `used_slots >= availableSlots`.

[M-05] Failure to verify USDC transfer results can lead to loss of funds

Severity

Impact: High

Likelihood: Low

Description

The `RelayPolymarketSDK::transferUsdc()` function transfers USDC from one address to another and is used in multiple flows such as referral withdrawals, fee transfers, and USDC withdrawals. This function is called without checking whether the operation succeeds.

However, this function can fail silently (without throwing an exception) without stopping the flow. As a result, even if the USDC transfer transaction actually fails, the flow still continues running to the end. This can lead to the loss of funds for both the protocol and users.

```
const result = await sdk.transferUsdc({
    to: masterFeeAddress,
    amount: feeAmountBigInt,
});
```

For example, in the fee transfer flow, the USDC is transferred from the user to the master fee address. If the transfer fails, the commission fee is still accounted for the beneficiaries, resulting in a loss of funds for the master fee address.

Recommendations

Ensure that the USDC transfer transaction succeeds before continuing the flow. The return value or transaction status should be checked, and the process should revert or stop if the transfer fails.

[M-06] `mapHistoryRow` unbound `this` in map() causes runtime TypeError

Severity

Impact: Medium

Likelihood: Medium

Description

`getCopyTradeHistory()` at `copy-trading.ts:1272` and `getCopyTradeHistoryByFollow()` at `copy-trading.ts:1386` pass `this.mapHistoryRow` as an unbound method reference to `.map()`:

```
// copy-trading.ts:1272
return records.map(this.mapHistoryRow);

// copy-trading.ts:1386
return records.map(this.mapHistoryRow);
```

The `mapHistoryRow` method at lines 1672-1673 calls `this.mapHistoryRowWithLeaderName()`:

```
private mapHistoryRow(row: typeof copyTradeHistory2.$inferSelect): CopyTradeHistoryRecord {
    return this.mapHistoryRowWithLeaderName(row, null);  // `this` is undefined when unbound
}
```

When JavaScript invokes an unbound method reference in `.map()`, `this` is `undefined` (strict mode). The call to `this.mapHistoryRowWithLeaderName()` throws `TypeError: Cannot read properties of undefined (reading 'mapHistoryRowWithLeaderName')`.

Contrast with line 1366, which correctly uses an arrow function:

```
return records.map(r => this.mapHistoryRowWithLeaderName(r.history, r.leaderName));  // @audit correct
```

Recommendations

Change to function: `records.map(r => this.mapHistoryRow(r))`.

[M-07] Non-atomic commission balance operation enables race conditions

Severity

Impact: Medium

Likelihood: Medium

Description

The `addPendingCommission()` and `resetPendingCommission()` methods in `packages/database/src/services/users.ts:343-374` operate independently without coordination, creating a race condition window:

```
// addPendingCommission uses atomic SQL addition (good)
pendingCommission: sql`COALESCE(${users.pendingCommission}, 0) + ${amount.toString()}`

// resetPendingCommission unconditionally sets to 0
pendingCommission: '0'
```

While `addPendingCommission` uses an atomic SQL increment (safe against concurrent adds), `resetPendingCommission` unconditionally sets the balance to '0'. This creates a race condition:

1. A trade completes and `addPendingCommission(userId, 10)` is called.
2. Simultaneously, the user requests a withdrawal and `resetPendingCommission(userId)` is called.
3. Depending on execution order:
4. If add executes first, then reset: commission is lost (add's $10 wiped by reset).
5. If reset executes first, then add: commission is not withdrawn (reset's $0 overwritten by add).

Recommendations

1. Use `resetPendingCommission` with an atomic swap: read the current value and return it, then use that value for the withdrawal job, all in a single transaction.
2. Add validation that `amount > 0` in `addPendingCommission`.
3. Consider using `UPDATE ... SET pendingCommission = 0 WHERE pendingCommission > 0 RETURNING pendingCommission` for atomic reset-and-read.

[M-08] Promo claim updates state without validation

Severity

Impact: Medium

Likelihood: Medium

Description

The `claimPromo()` method in `packages/database/src/services/kol-promo.ts:122-149` updates the user promo status to `'claimed'` without validating that:

1. **The current status is** `'pending'`: The WHERE clause only filters by `userPromos.id`, not by `status = 'pending'`. If the promo is already `'claimed'` or `'unlocked'`, the method will overwrite the existing `claimedAt`, `unlockAt`, `claimTxHash`, and `claimedAmount` values, resetting the cooldown window and potentially overwriting a valid transaction hash.

```
// kol-promo.ts:127-137
const [updated] = await this.db.update(userPromos)
    .set({
        status: 'claimed',
        claimedAt: now,
        unlockAt,           // Resets 3-day cooldown
```

```
        claimedAmount: amount,
        claimTxHash: txHash, // Overwrites prior valid txHash
        updatedAt: now,
    })
    .where(eq(userPromos.id, userPromoId))  // No status check!
    .returning();
```

1. **The `txHash` is a valid, non-empty transaction hash**: The worker (`promo-claim.worker.ts`) passes `result.transactionHash || ''` ; an empty string is accepted and stored if the transfer fails silently. The ERC20 `transfer()` function can return `false` without reverting, and the relay SDK does not validate the return value. This means the promo can be marked as `'claimed'` with an empty `claimTxHash` while no funds were actually transferred.

The same pattern affects `markWithdrawn()` in `commissions.ts:107-124` ; it has a status guard (`status = 'credited'`) but does not validate the `txHash` parameter.

Recommendations

1. Add `eq(userPromos.status, 'pending')` to the WHERE clause in `claimPromo()` (similar to the guard comment that exists in the code but was not implemented).
2. Validate that `txHash` is a non-empty, valid 66-character hex string (0x-prefixed) before updating the database.
3. Verify the on-chain transaction receipt before marking the promo as claimed.

[M-09] Unencrypted database communication

Severity

Impact: Medium

Likelihood: Medium

Description

In `packages/database/client/index.ts` , the database client connection is configured without SSL, leaving database communication unencrypted. This exposes sensitive data to potential interception in man-in-the-middle attacks, particularly concerning given the nature of the data being stored and transmitted.

```
export const createDbClient = (connectionString: string, options?: { max?: number }) => {
  const client = postgres(connectionString, {
    max: options?.max ?? 10,
    idle_timeout: 20, // Release idle connections after 20s to free resources
    connect_timeout: 10, // Fail fast if can't get connection in 10s
    max_lifetime: 60 * 30, // Rotate connections every 30 minutes
    prepare: true, // Use prepared statements for better performance (direct PostgreSQL)
  });
  return drizzle(client, { schema });
};
```

Recommendations

Enable SSL/TLS encryption for all database connections by setting the appropriate SSL configuration.

```
export const createDbClient = (connectionString: string, options?: { max?: number }) => {
  const client = postgres(connectionString, {
    max: options?.max ?? 10,
    idle_timeout: 20, // Release idle connections after 20s to free resources
    connect_timeout: 10, // Fail fast if can't get connection in 10s
    max_lifetime: 60 * 30, // Rotate connections every 30 minutes
    prepare: true, // Use prepared statements for better performance (direct PostgreSQL)
+   ssl: true
  });
  return drizzle(client, { schema });
};
```

[M-10] Migration fails due to non-existent serial type

Severity

Impact: Medium

Likelihood: Medium

Description

In `schema.ts`, `copy_trading_follows.id` is defined as:

```
id: serial('id').primaryKey(),
```

The table was originally created in migration `0011_wise_marauders.sql` with `id` as **uuid**. Any migration that tries to change this column to match the current schema (e.g. `ALTER COLUMN "id" TYPE serial`) will **always fail** in PostgreSQL with:

`type serial does not exist`

In PostgreSQL, `serial` is not a real type. It is only valid in `CREATE TABLE`, where it expands to `integer` plus a sequence. In `ALTER TABLE ... ALTER COLUMN ... TYPE serial`, the type name `serial` is invalid — PostgreSQL does not accept it.

So the migration that would align the database with the schema cannot run.

Recommendations

Consider keeping the column as UUID in the database and changing the schema to `uuid('id').defaultRandom().primaryKey()` so no type change is needed, or deleting the previously generated migrations (if they are not already deployed) so that the serial type is applied successfully.

Low findings

[L-01] pauseSubscription() overwrites pause reason without check

Description

`pauseSubscription(followId, reason)` sets `isActive = false`, `autoStoppedAt`, and `autoStoppedReason` with only `WHERE id = followId`. It does not check that the follow is currently **active** (`isActive = true`). So it can run on an already paused follow and overwrite the existing pause time and reason.

Recommendations:

Consider adding a precondition to the `WHERE` clause so the update only runs on active follows.

[L-02] `reactivateFollow()` does not verify the follow is paused

Description

`reactivateFollow(followId)` sets `isActive = true` and clears all failure/auto-stop state (`consecutiveFailures`, `lastFailureReason`, `notificationsSilenced`, `autoStoppedAt`, `autoStoppedReason`) with only `WHERE id = followId`.

It does not check that the follow is actually paused (`isActive = false`).

Recommendations:

Consider adding preconditions to the `WHERE` clause so the update only runs on paused follows.

[L-03] No application path to set user ban `isBanned`

Description

The application enforces a user ban when `users.is_banned` is true (bot blocks and shows a message) but provides no code path to set `isBanned` to true.

Banning is only possible via direct database access.

Recommendations:

Consider adding a supported way to set the ban flag, for example: a `UserService.updateBanned(userId, banned: boolean)` that is used by an admin-only script or command.

[L-04] Commission creation default status vs schema

Description

The schema default for commission status is `'pending'`, but the create function uses `params.status || 'credited'`, so when status is not provided, the DB default is overridden in application code and new commissions are created as `'credited'` instead of `'pending'`.

Recommendations:

Consider aligning behavior with the schema and product rules: either default to `'pending'` in code when `params.status` is omitted, or change the schema default to match the intended default and document it.

[L-05] Wallet mapping omits `deletedAt`

Description

The wallet row-to-DTO mapping (`mapRowToWallet`) does not expose `deletedAt`. If wallets can be soft-deleted, callers cannot see the deletion state from the mapped object.

Recommendations:

Consider adding `deletedAt` to the wallet DTO and mapping.

[L-06] User mapping omits copy trading field

Description

The user row-to-DTO mapping (`mapRowToUser`) does not include copy-trading fields (`copyTradeInsufficientBalanceCount`, `copyTradePausedAt`, `copyTradePausedReason`).

Callers that load a “full” user never see these values.

Recommendations:

Consider extending the mapping and the returned user type to include the copy-trading fields so that the UI and logic that depend on pause state and counts have correct data.

[L-07] Soft delete does not update `updatedAt`

Description

The `softDeleteUser` update sets only `deletedAt`, not `updatedAt`. Auditing and “last modified” semantics are inconsistent with other updates that do not set `updatedAt`.

Recommendations:

Consider setting `updatedAt` to the current timestamp in the same update that sets `deletedAt` so that soft delete is treated like any other update.

[L-08] `telegramId` not unique in the wallets table

Description

The wallets table references `users.telegramId` but does not enforce uniqueness on `telegram_id`. Uniqueness is enforced in the service: `createWallet` in `wallets.ts` calls `getWalletByTelegramId` and, if a wallet exists, returns the existing one instead of inserting (idempotent create). So the normal create path preserves one wallet per telegram ID. Duplicates could still be introduced by other code paths, scripts, or direct DB access.

It is **not** enforced at the DB/schema level.

Recommendations:

Consider adding a unique constraint on `wallets.telegram_id` so the invariant is enforced at the DB level; run a one-off check for existing duplicates before applying.

[L-09] `referralCode` not unique in the users table

Description

The `referral_code` column on the users table is not declared unique. Referral codes are used to attribute referrals; if two users share the same code, attribution would be ambiguous or wrong. Uniqueness is enforced in the service only when generating codes: `generateUniqueReferralCode` in `users.ts` checks `findByReferralCode` and retries on collision.

It is **not** enforced at the DB/schema level.

Recommendations:

Consider adding a unique constraint on `referral_code` in the schema so the DB enforces uniqueness regardless of the caller.

[L-10] Potential rounding error in wei-to-token conversion

Description

When converting from the token amount in wei to the human-readable token amount, the code uses the formula:

```
Number(a) / 1e6
```

where `a` represents the token amount in wei.

This approach can lead to incorrect results when `a` is greater than `Number.MAX_SAFE_INTEGER`. Values exceeding `2^53 - 1` lose integer precision. As a result, converting large `BigInt` values to `Number` may produce rounding errors or inaccurate token amounts.

Recommendation: Use a safe formatting utility such as:

```
ethers.utils.formatUnits(a, 6)
```

[L-11] Copy trading actions lack authorization check

Description

In the copy trading handlers (`packages/telegram-ui-v2/src/handlers/copy-trading.ts`), multiple action handlers (such as pause, resume, mute, unmute, and edit operations) follow a pattern where they extract a `followId` from the callback data, perform actions on the subscription via the service, and then refresh the view.

However, these handlers do not verify that the provided `followId` belongs to a subscription owned by the current user. This creates a potential vulnerability where a user can manipulate other users' copy trading subscriptions by crafting callback data with arbitrary `followId` values. The system trusts the client-provided identifier without performing proper authorization checks.

Example handler:

```
bot.action(/^copy_trade_pause_(\d+)$/, async (ctx: PolygunContext) => {
    await ctx.answerCbQuery('Paused');

    const followId = parseInt(ctx.match[1], 10);
    const copyTradingService = getCopyTradingService(ctx);

    await copyTradingService.pauseFollow(followId);
    ...
});
```

Service implementation:

```
async pauseFollow(followId: number): Promise<void> {
    try {
        await this.db.update(copyTradingFollows)
            .set({ isActive: false, updatedAt: new Date() })
            .where(eq(copyTradingFollows.id, followId));

        logger.info('Follow paused', { followId });
    } catch (error) {
        throw new DatabaseError(`Failed to pause follow: ${error instanceof Error ?
error.message : 'Unknown error'}`);
    }
}
```

Recommendation: Do not rely solely on `followId`. Always verify that the subscription belongs to the caller.

[L-12] Pending commission can be restored after successful USDC transfer

Description

In the referral withdrawal worker (`referral-withdraw.worker.ts`), the pending commission is restored to the user if an exception is thrown in the `try` block, even if the USDC transfer has already succeeded.

As a result, if any logic executed **after** the USDC transfer throws an exception, the user's pending commission balance is restored, even though the funds have already been transferred. This allows the user to withdraw the referral fee again.

```
try {
       ...
        // Execute transfer from master's Gnosis Safe to user's wallet
        const result = await sdk.transferUsdc({
            to: destinationAddress,
            amount: amountInUnits,
        });
        ...
        await commissionService.markWithdrawn(userId, txHash);

        // Send success notification
        await sendReferralWithdrawSuccessNotification(config.botToken, chatId, {
            amount: amount.toFixed(2),
            txHash,
        });

        return {
            status: 'success',
            txHash,
        };
    } catch (error) {
       ...
        // Restore the pending commission balance on failure
        try {
            await userService.addPendingCommission(userId, amount);
        } catch (restoreError) {
            ...
        }
        ...
    }
},
```

This creates a double-withdrawal risk:

- The USDC transfer succeeds.
- A later operation throws an exception.
- The pending commission is restored.
- The user can withdraw the same commission again.

Recommendation: Only restore the pending commission balance if the USDC transfer fails.

[L-13] Missing `feePercent` in fee transfer job data

Description

In the limit order processor worker (`limit-order-processor.worker.ts`), when adding a new job to the fee transfer queue, the job data does not include the `feePercent` field. As a result, the fee transfer worker receives this field as `undefined` and stores a new record in the `fee_transfers` table with `feePercent` set to `null` .

```
await feeTransferQueue.add(
    'fee-transfer',
    {
        telegramId: limitOrder.telegramId,
        safeAddress: maker,
        tradeType: 'limit_fill',
        tradeId: orderHash,
        marketId: limitOrder.marketId,
        tokenId: limitOrder.tokenId,
        tradeAmount: limitOrder.originalSize ? parseFloat(limitOrder.originalSize) :
undefined,
        feeAmount: parseFloat(limitOrder.feeAmount),
        fromAddress: maker,
        toAddress: config.masterFeeAddress,
        sourceJobId: job.id,
        limitOrderId: limitOrder.id, // Mark order as fee taken after success
    } as FeeTransferJobData,
```

Recommendation: Add the `feePercent` field to the job data when enqueuing the fee transfer job.

[L-14] Incorrect price calculation in limit order processor

Description

In the limit order processor worker (limit-order-processor.worker.ts), there is a logic error in the price calculation for sell orders. When determining the `usdcAmount` for price computation, the code incorrectly uses `takerAssetId` instead of `takerAmountFilled` for sell orders. The `takerAssetId` is an asset identifier, not an amount, causing the price calculation to use an irrelevant value. This results in incorrect fill prices being computed and subsequently displayed in user notifications.

```
const usdcAmount = isBuy ? BigInt(makerAmountFilled) : BigInt(takerAssetId);
const sharesAmount = isBuy ? BigInt(takerAmountFilled) : BigInt(makerAmountFilled);
const fillPrice = sharesAmount > 0n
    ? (Number(usdcAmount) / Number(sharesAmount)) * 100 // Price in cents
    : 0;
```

Recommendation: Fix the price calculation logic by replacing `BigInt(takerAssetId)` with `BigInt(takerAmountFilled)` for sell orders.

[L-15] Limit order may be marked fully filled due to accounting mismatch

Description

Users may be charged a fee even when their limit order is not fully filled due to incorrect accounting logic.

More specifically, the limit order processor worker (`packages/queue/src/workers/limit-order-processor.worker.ts`) processes `OrderFilled` events for limit orders. The processor determines whether an order is fully filled based on the current filled amount (`currentFilled`) and the new fill amount (`fillAmount`). If the order is considered fully filled, the fee is transferred from the user wallet to the master wallet. Otherwise, no fee is charged.

```
// Step 3: Calculate fill amounts from event
const isBuy = limitOrder.side === 'BUY';
const fillAmountRaw = isBuy
    ? BigInt(takerAmountFilled) // shares filled
    : BigInt(makerAmountFilled); // shares filled
const fillAmount = Number(fillAmountRaw) / 1e6;
...
// Step 5: Determine if fully filled (99% threshold)
const originalSize = parseFloat(limitOrder.originalSize || '0');
const currentFilled = parseFloat(limitOrder.filledSize || '0');
const newFilledSize = currentFilled + fillAmount;
const isFullyFilled = originalSize > 0 && newFilledSize >= originalSize * 0.99;

// Step 6: Update fill tracking in database
try {
    await limitOrderService.updateFill(limitOrder.id, fillAmount, isFullyFilled);
} catch (error) {
    ...
}
...
```

However, `newFilledSize = currentFilled + fillAmount` can be incorrect because `fillAmount` represents the number of shares, while `currentFilled` may represent the amount of USDC in the case of a limit buy order. This results in inconsistent units being added together.

As a result, `newFilledSize` may be greater than the actual filled value, causing `isFullyFilled` to be evaluated as true even though the order is not fully filled on-chain.

Example Scenario

1. A limit buy order is created with `originalSize = 200 USDC` and `limitPrice = 0.8` (`limitPrice` ranges from 0.01 to 0.99).
2. A user fills the order with 200 shares and receives 160 USDC.
3. Due to inconsistent unit handling, `newFilledSize` is incorrectly treated as 200 instead of 160.
4. As a result, `isFullyFilled` is evaluated as `true`.

Consequently, the user is charged a fee even though the limit order has not been fully filled according to the incorrect accounting logic.

Recommendations

Ensure that `fillAmount` and `currentFilled` use the same unit (either shares or USDC) before performing the comparison.

For example, `fillAmount` should be calculated consistently using:

```
ethers.utils.formatUnits(BigInt(makerAmountFilled), 6)
```

so that the filled amount is tracked in the same unit as `originalSize`.

[L-16] Multiple unbounded queries risk OOM during backlog processing

Description

Several methods fetch all matching records with no `LIMIT`, creating an Out Of Memory (OOM) risk during backlogs:

The table outlines several methods within the codebase that are at risk of causing Out Of Memory (OOM) errors during backlog processing due to the absence of a `LIMIT` clause in their queries. Each entry specifies a method name, the corresponding file where it is implemented, the line number of the implementation, and the database table with which the method interacts.

The first method, `getPendingTransfers()`, is located in the file `fee-transfers.ts` at line 248 and interacts with the `fee_transfers` table. Next, `getPendingErrors()` can be found in `fee-errors.ts` at line 63, and it queries the `fee_processing_errors` table. The method `getByTelegramId()` is also in `fee-errors.ts`, specifically at line 168, and it accesses the same `fee_processing_errors` table.

Additionally, the method `findByLimitOrderId()` is implemented in `fee-errors.ts` at line 93, and it too queries the `fee_processing_errors` table. The method `getUnfilledByTelegramId()` is located in `limit-orders.ts` at line 142, interacting with the `limit_orders` table.

Furthermore, `getUniqueCopyTraders()` is found in `copy-trading.ts` at line 1801, and it queries the `copy_trade_history_2` table, which contains over 50 million rows, presenting a significant risk for OOM errors. Lastly, the method `getAllUsersWithWallets()` is located in `users.ts` at line 380, and it accesses the `users` table.

This information highlights the potential risks associated with these methods, emphasizing the need for implementing appropriate limits to mitigate the Out Of Memory (OOM) risk during backlog processing.

Recommendations

Add reasonable `LIMIT` clauses to all batch-processing queries.

[L-17] Database client has no statement timeouts

Description

The database client at `client/index.ts:6-12` has no query timeout:

```
const client = postgres(connectionString, {
    max: options?.max ?? 10,
    idle_timeout: 20,
    connect_timeout: 10,
    max_lifetime: 60 * 30,
    prepare: true,
    // Missing: statement_timeout or query_timeout
});
```

With a pool max of 10, a few long-running queries (e.g., `getUniqueCopyTraders`) can exhaust all connections, causing cascading timeouts across the entire application.

Recommendations

Add `statement_timeout` (e.g., 30 seconds) to the client configuration.

[L-18] `incrementCompletedTrades` swallows errors

Description

`incrementCompletedTrades()` at `kol-promo.ts:187-190` swallows all errors:

```
} catch (error) {
    logger.error({ error, telegramId }, 'Failed to increment completed trades');
    return null;  // Same return as "user has no active promo"
}
```

The caller has no way to distinguish "no promo" from "DB failure." The trade is already executed, but the user's `completedTrades` counter is not incremented. If this happens on the qualifying trade that should unlock the promo, the user must make an extra trade.

Recommendations

Throw the error so the caller can retry or alert.

[L-19] No check constraints on 12+ status enum text columns

Description

Multiple columns across the schema use `text` type for enumerated values without any database-level `CHECK` constraints:

The table outlines a security audit finding related to the use of `text` type columns for enumerated values across various database tables. Specifically, it highlights the absence of database-level `CHECK` constraints on these columns, which can lead to potential data integrity issues.

The following details are provided for each identified column:

- In the **users** table, the `tradingMode` column is expected to contain one of the following values: 'cautious', 'standard', or 'expert'.
- The **referralCommissions** table includes a `status` column that should reflect one of these values: 'pending', 'credited', or 'withdrawn'.
- The **temporaryWallets** table has a `status` column where the expected values are 'available', 'claimed', or 'creating'.
- In the **limitOrders** table, the `side` column is expected to have either 'BUY' or 'SELL' as its values.
- The **feeProcessingErrors** table's `status` column should indicate 'pending', 'resolved', or 'failed'.
- The **copyTradingFollows** table contains a `mode` column that is expected to have one of the following values: 'proportional', 'fixed', or 'percentage'.
- The **feeTransfers** table includes a `status` column that should reflect 'pending', 'success', or 'failed'.
- Lastly, the **userPromos** table has a `status` column where the expected values are 'pending', 'claimed', or 'unlocked'.

The absence of `CHECK` constraints on these columns raises concerns regarding the enforcement of valid data entries, which is critical for maintaining data integrity within the database.

Any application bug or SQL injection can write arbitrary values, corrupting status-based queries throughout the system (fee processing, copy trading, promotion redemption).

Recommendations

Add `CHECK` constraints or use PostgreSQL `enum` types for status columns.

[L-20] `fee_transfers` table has no deduplication on `tradeId`

Description

The `feeTransfers` schema at `schema.ts:384-417` defines no unique constraint on `tradeId` (or any combination including it). The `tradeId` column is even nullable (line 390):

```
tradeId: text('trade_id'),  // nullable, no unique constraint
```

`FeeTransferService.create()` at `fee-transfers.ts:56-92` inserts unconditionally with no idempotency guard, no `ON CONFLICT`, no pre-check. If the same trade triggers fee collection twice (BullMQ job retry, network timeout where the first insert succeeded), two fee transfer records are created for the same trade. Both get `status: 'pending'` and both will be processed, charging the user the fee twice.

Recommendations

1. Add a `UNIQUE` constraint on `(trade_id, telegram_id)` or `trade_id` where it is not null.
2. Use `INSERT ... ON CONFLICT DO NOTHING` in `create()`.

[L-21] FindOrCreateByTelegramId prevents reregistration of soft-deleted users

Description

The `findOrCreateByTelegramId()` method at `users.ts:87-112` searches for users with `deletedAt IS NULL`:

```
const existingUser = await this.db.query.users.findFirst({
    where: (users, { eq, and, isNull }) =>
        and(eq(users.telegramId, telegramId), isNull(users.deletedAt)),
});
```

If no active user is found, it attempts to insert. However, the `telegramId` column has a global `UNIQUE` constraint at `schema.ts:5`:

```
telegramId: text('telegram_id').unique().notNull(),
```

If a user was previously soft-deleted (their row has `deletedAt` set but `telegramId` is still present), the `findFirst` returns null (filtered by deletedAt), but the INSERT fails with a unique constraint violation because the soft-deleted row's `telegramId` already exists.

Recommendations

1. Change the unique constraint to a partial index: `UNIQUE WHERE deleted_at IS NULL`.
2. Or in `findOrCreateByTelegramId`, check for soft-deleted users and re-activate them instead of creating a new row.

[L-22] Timestamp columns use mixed timezone handling

Description

`dailyStatsSnapshots` (`schema.ts:452-453`) uses `{ withTimezone: true }` while ALL other tables use naive `timestamp()` without timezone. Cross-table queries require implicit timezone conversion that depends on session settings.

Recommendations

Standardize all timestamp columns to `timestamp('...', { withTimezone: true })`.

[L-23] `setReferrer` allows changing referrer after initial set

Description

The `setReferrer()` method in `packages/database/src/services/users.ts:315-338` does not check if `referredById` is already set:

```
// users.ts:326-332
await this.db.update(users)
    .set({
        referredById: referrerId,
        referralLevel: newLevel,
    })
    .where(eq(users.id, userId));  // No check: referredById IS NULL
```

A user's referrer can be changed at any time, moving them between referral chains. Past commissions remain with the old referrer, but future commissions go to the new one, creating inconsistent attribution. Network statistics (`getNetworkStats`) also become incorrect.

Recommendations

Add `AND referredById IS NULL` to the WHERE clause. Once set, the referrer is immutable.

[L-24] Missing input validation at database service layer

Description

Multiple database service methods accept user-influenced parameters without bounds validation, relying entirely on caller-side validation (UI handlers). This creates a defense-in-depth gap where any bypass of UI validation (direct API call, new caller, compromised handler) results in invalid data entering the database.

Key examples:

1. `updateFollowSettings()` **in** `copy-trading.ts:292-349` : Accepts `multiplier` , `fixedAmount` , `percentageAmount` , `dailyLimit` , `singleTradeLimit` as raw numbers with no bounds checking. Negative multipliers could invert trades; zero fixed amounts could cause division by zero.
2. `updateTradingFeePercent()` **in** `users.ts:500-510` : Accepts any numeric value including negative numbers. A negative fee percent would effectively pay users to trade instead of charging them.
3. `CommissionService.create()` **in** `commissions.ts:39-66` : Accepts `commissionPercent` , `commissionAmount` , and `tradeFeeAmount` with no positivity validation. Negative commission amounts would create phantom balances.
4. `referralPercents` **field**: Stored as unparsed JSON text (`'[0.25, 0.05, 0.03]'`). The parser in `trading-utils.ts` performs `JSON.parse()` and checks array length >= 3, but does not validate that values are between 0 and 1 (0% to 100%). Values greater than 1 would cause commission amounts exceeding the trade fee.

Recommendations

1. Add bounds validation at the service layer as a safety net: `multiplier > 0` , `fixedAmount > 0` , `feePercent >= 0` , `commissionAmount >= 0` .
2. Add a validation function for `referralPercents` that ensures all values are in [0, 1].
3. Consider using database-level CHECK constraints for numeric fields.

[L-25] HasPromoRestriction fails open on database error

Description

The `hasPromoRestriction()` method in `packages/database/src/services/kol-promo.ts:265-308` returns `{ restricted: false }` when a database error occurs:

```
async hasPromoRestriction(userId: string): Promise<PromoRestriction> {
    try {
        // ... restriction logic ...
    } catch (error) {
        logger.error({ error, userId }, 'Failed to check promo restriction');
        // Default to not restricted on error to avoid blocking users
```

```
        return { restricted: false };  // FAIL-OPEN
    }
}
```

This "fail-open" pattern means that any database connectivity issue (timeout, connection pool exhaustion, network partition) will cause promo-restricted users to gain full access. Promo restrictions exist to prevent users from withdrawing promo-funded USDC before completing required trades and cooldown periods. During a database outage, these restrictions are silently bypassed.

This is particularly concerning because:

- Promo restrictions gate access to private key export (checked in settings handlers).
- A determined attacker could trigger database errors (e.g., via resource exhaustion) to bypass restrictions.

Recommendations

1. Change to a fail-closed pattern: return `{ restricted: true, reason: 'error_checking_restriction' }` on database errors.
2. At a minimum, log the fail-open event at `warn` level for monitoring and alerting.

[L-26] Missing unique constraint allows duplicate promos

Description

The `createUserPromo()` method in `packages/database/src/services/kol-promo.ts` creates a new `userPromos` record without checking for existing promos from the same KOL for the same user. While `getActiveUserPromo()` checks for promos with `status != 'unlocked'`, the check is performed at the handler level, not atomically in the database.

The `userPromos` table in `schema.ts` defines no unique constraint on `(userId, kolPromoId)`. This means:

1. Two concurrent requests for the same user and promo reference code both pass the handler-level check.
2. Both call `createUserPromo(userId, kolPromoId)`.
3. Both INSERT statements succeed, creating duplicate promo claims.
4. The user gets 2 times the promo amount; `usedSlots` is incremented twice.

Recommendations

1. Add a `UNIQUE(userId, kolPromoId)` constraint to the `userPromos` table.
2. Handle the unique constraint violation error in `createUserPromo()` gracefully.

[L-27] `Math.random()` used for share codes and referral codes

Description

Two separate code paths use `Math.random()` to generate relevant identifiers:

1. **Copy-trading share codes** in `packages/database/src/services/copy-trading.ts:129-136`:

```
function generateShareCode(): string {
    const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789';
    let code = '';
    for (let i = 0; i < 8; i++) {
        code += chars.charAt(Math.floor(Math.random() * chars.length));
    }
    return code;
}
```

1. **Referral codes** in `packages/database/src/services/users.ts` (same `Math.random()` pattern).

`Math.random()` uses a non-cryptographic PRNG (V8's xorshift128+). Given:

- Share codes are used to join copy-trading subscriptions (act as an access token).
- Referral codes are tied to financial incentives (commission percentages).
- The keyspace is only 62^8 ≈ 218 trillion values, but `Math.random()` PRNG state has only 2^128 internal state and can be reconstructed from a few observed outputs.

Recommendations

1. Replace `Math.random()` with `crypto.randomBytes()` or `crypto.randomInt()` for generating both share codes and referral codes.
2. Example: `crypto.randomBytes(6).toString('base64url').slice(0, 8)` provides 48 bits of cryptographic randomness.

[L-28] `updateFollowSettings` does not check `deletedAt`

Description

The `updateFollowSettings()` method in `packages/database/src/services/copy-trading.ts:292-349` updates follow relationship settings using only the `followId` in the WHERE clause:

```
// copy-trading.ts:341-343
await this.db.update(copyTradingFollows)
    .set(updateData)
    .where(eq(copyTradingFollows.id, followId));
    // No check for deletedAt IS NULL!
```

The `copyTradingFollows` table uses soft deletes via the `deletedAt` column (`schema.ts`). Other query methods like `getActiveFollowersWithSafeStatus()` correctly filter by `deletedAt IS NULL`, but `updateFollowSettings` does not.

This means:

1. A user unfollows a trader (setting `deletedAt` to the current timestamp).
2. Through a race condition or API manipulation, `updateFollowSettings` is called on the soft-deleted follow.
3. The follow's settings are updated (mode, multiplier, limits), but it remains "deleted".
4. If a bug or future code change inadvertently reactivates deleted follows, the modified settings take effect.

The same issue exists in `pauseFollow()` and `resumeFollow()`. They operate on `followId` without checking `deletedAt`.

Recommendations

1. Add `isNull(copyTradingFollows.deletedAt)` to the WHERE clause in `updateFollowSettings`, `pauseFollow`, and `resumeFollow`.
2. Check affected rows count and throw an error if zero rows were updated.

[L-29] Unsafe usage of `sql.raw()` for `INTERVAL` in getTotalFees

Description

`fee-transfers.ts` constructs the `INTERVAL` value using `sql.raw()`:

`sql.raw()` emits its argument directly into the query without parameterization. `intervals` is a local dictionary and the period parameter is typed as `'hour' | 'day' | 'week' | 'month'`, so TypeScript prevents direct injection. However, if this function is called from a JavaScript caller or if `period` is cast from an external string, the raw value is injected unescaped into the SQL. Postgres `INTERVAL` syntax accepts arbitrary input and errors would expose query structure in the `DatabaseError` message.

Fix:

```
- gte(feeTransfers.createdAt, sql`NOW() - INTERVAL '${sql.raw(intervals[period])}'`)
+ gte(feeTransfers.createdAt, sql`NOW() - ${intervals[period]}::interval`)
```

[L-30] Schema enforces uniqueness only via application-level checks

Description

Four uniqueness invariants are enforced only in application code with no corresponding UNIQUE constraint at the database level:

- `users.referralCode`
- `wallets.telegramId`
- `copyTradeHistory2.followerTelegramId`
- `copyTradeHistory2.leaderAddress`
- `userPromos.userId`

Any direct DB access, migration side effect, or application bug that bypasses the service layer will silently insert duplicates. Add all four to have `unique()` so the DB enforces the invariant independently of the ORM layer.

[L-31] Inconsistent error handling at `findByTokenId`

Description

However, that is not the case with `findByTokenId` as it simply warns via logger and returns `null`:

```
catch (error) {
            logger.warn('Failed to find market by token ID', { tokenId, error });
            return null;
        }
```

Fix: Maintain consistency by throwing an error on failure at `findByTokenId()`:

```
catch (error) {
            throw new DatabaseError(
                `Failed to find market by  token ID: ${error instanceof Error ?
error.message : 'Unknown error'}`
            );
        }
```

[L-32] `UserService.withRetry()` implements linear not exponential backoff

Description

However, the implementation is linear `(delayMs × attempt)`. Under sustained DB pressure, retries arrive faster than intended, increasing load instead of backing off.

It should be `delayMs * 2 ** (attempt - 1)` as specified in your document.

[L-33] Kol-promo.ts checkAndUnlockPromo missing status guard on `UPDATE`

Description

Two concurrent calls that both read `status='claimed'`, with conditions met, and both issue this `UPDATE` in the `checkAndUnlockPromo()` function. The second `UPDATE` matches with the `'unlocked'` row; however, due to no status guard, it returns 1 row affected, and both callers receive `true`. Both trigger the unlock notification and any downstream reward disbursement.

```
await this.db.update(userPromos)
    .set({ status: 'unlocked', updatedAt: now })
    .where(eq(userPromos.id, userPromo.id));
```

To fix this, implement a more robust defense to `kol-promo.ts::checkAndUnlockPromo()`:

```
.where(and(eq(userPromos.id, userPromo.id), eq(userPromos.status, 'claimed')))
```

[L-34] Auto claim cron includes banned users

Description

The method used by the auto-claim cron job selects all users with a non-null `gnosisSafeAddress` and no `deleted_at`, but does not filter on `is_banned` = false:

```
.where(and(
    sql`${users.gnosisSafeAddress} IS NOT NULL AND ${users.gnosisSafeAddress} != ''`,
    isNull(users.deletedAt)
    //@audit isBanned not filtered
))
```

Banned users are included in auto-claim cycles, triggering on-chain transactions on their behalf. Although banned users cannot claim it directly as the UI-level claim function will not be shown.

Recommended Fix

Add a check to filter the banned users - `eq(users.isBanned, false)` to the `WHERE` clause.

[L-35] PostgreSQL rounds decimal(18, 6) inserts causing potential overcharges

Description

Values written to columns declared as `decimal(18, 6)` (or `numeric(18, 6)`) with **more than 6 fractional digits** are **rounded** by PostgreSQL to 6 decimal places on insert. The database does **not** truncate. Application code passes values via `.toString()` (or similar) without normalizing scale; the rounding happens in the database.

This behaviour applies to **all** inserts in DB services that write to such columns; fee errors, fee transfers, limit orders, commissions, and any other tables with a fixed-scale decimal.

Example: inserting `5.5123459` into a `decimal(18, 6)` column stores `5.512346` (round half-up), not `5.512345`.

PostgreSQL's behaviour is defined in the Numeric Types documentation:

> If the scale of a value to be stored is greater than the declared scale of the column, **the system will round the value to the specified number of fractional digits**.
> — PostgreSQL 18: 8.1.2 Arbitrary Precision Numbers

So any value with more than 6 fractional digits is coerced by rounding when the column scale is 6.

The codebase does not truncate or round in application code before insert for most fee/amount fields; it sends the full number string, so the only coercion is PostgreSQL's rounding.

This results in the stored fee (or other) amounts being slightly higher than the exact value computed in code.

Proof of Concept:

```
/**

 * POC: PostgreSQL rounds numeric(18,6) on insert; it does not truncate.
 * See findings/M-01-postgres-decimal-rounding-on-insert.md
 *
 * Refs:
 * - https://www.postgresql.org/docs/current/datatype-numeric.html
 *   "If the scale of a value to be stored is greater than the declared scale of the column,
 *    the system will round the value to the specified number of fractional digits."
 */
import { describe, it, expect, beforeAll, afterAll } from 'vitest';
import { sql } from 'drizzle-orm';
import { createDbClient } from '../client/index.js';

const connectionString = process.env.DATABASE_URL;

describe('PostgreSQL decimal(18,6) rounding on insert (POC)', () => {
  const db = connectionString ? createDbClient(connectionString) : null;
```

```
  beforeAll(async () => {
    if (!db) return;
    await db.execute(sql`CREATE TEMP TABLE IF NOT EXISTS _poc_decimal (fee_amount
numeric(18,6) NOT NULL)`);
  });

  it('rounds value with 7+ fractional digits to 6 decimals (half-up), does not truncate',
async () => {
    if (!db) {
      console.warn('Skipped: set TEST_DATABASE_URL or DATABASE_URL to run');
      return;
    }

    // Value with 7 fractional digits; 9 in 7th place → round up 5.5123459 → 5.512346
    const valueToInsert = 5.5123459;
    const result = await db.execute(sql`
      INSERT INTO _poc_decimal (fee_amount) VALUES (${valueToInsert.toString()})
      RETURNING fee_amount
    `);
    const rows = result as unknown as Array<{ fee_amount: string }>;
    const stored = rows[0]?.fee_amount;

    expect(stored).toBeDefined();
    // PostgreSQL rounds: 5.5123459 → 5.512346 (not 5.512345)
    expect(stored).toBe('5.512346');
    expect(stored).not.toBe('5.512345');
  });
});
```

Recommendations

Consider normalizing scale in application code **before** insert by **rounding down** (truncating) to the column's scale. That way, the stored value never exceeds the exact computed value, and the DB no longer rounds up. For columns with scale 6, truncate to 6 fractional digits and pass a string, e.g.:

```
// Truncate to 6 decimal places (round down); then pass to insert
const feeAmountStr = (Math.floor(params.feeAmount * 1e6) / 1e6).toString();
// e.g. 5.5123459 → "5.512345" (not 5.512346)
```

[L-36] `uniqueFollow` index is not a unique constraint

Description

The `copyTradingFollows` table definition in `packages/database/src/schema.ts:267` defines what appears to be a uniqueness constraint but is actually a regular (non-unique) index:

```
// schema.ts:260-268
}, (table) => ({
    followerIdx: index('idx_copy_follows_follower').on(table.followerTelegramId),
    leaderAddrIdx: index('idx_copy_follows_leader_addr').on(table.leaderAddress),
    activeIdx: index('idx_copy_follows_active').on(table.isActive),
```

```
    leaderActiveIdx: index('idx_copy_follows_leader_active').on(table.leaderAddress,
table.isActive, table.deletedAt),
    // Ensure follower can only follow same leader once
    uniqueFollow: index('idx_copy_follows_unique').on(table.followerTelegramId,
table.leaderAddress),
    //            ^^^^^ This is index(), NOT uniqueIndex()!
}));
```

The comment says "Ensure follower can only follow the same leader once," but `index()` does not enforce uniqueness — only `uniqueIndex()` does. The application-level check in `follow()` at line 175 (`getFollowByLeaderAddress`) provides a TOCTOU-vulnerable soft check, but two concurrent `follow()` calls can both pass the check and insert duplicate records.

Duplicate follow relationships cause every trade by the leader to generate multiple copy-trade jobs for the same follower, multiplying the follower's trade size beyond their intended configuration.

Recommendations

1. Change `index('idx_copy_follows_unique')` to `uniqueIndex('idx_copy_follows_unique')` in the schema definition.
2. This requires a database migration. After migration, add appropriate error handling for the unique constraint violation in the `follow()` method.

[L-37] Self copy trading allows users to amplify own trades into infinite loop

Description

The `follow()` method in `packages/database/src/services/copy-trading.ts:153-222` does not check whether the follower's Telegram ID matches the leader's Telegram ID. A user can follow their own wallet address:

```
// copy-trading.ts:153-184
async follow(params: FollowParams): Promise<CopyTradingFollow> {
    const {
        followerTelegramId, leaderAddress, leaderTelegramId, ...
    } = params;
    // ...
    // Check if already following THIS address (but no check for self)
    const existing = await this.getFollowByLeaderAddress(followerTelegramId, leaderAddress);
    if (existing) {
        throw new DatabaseError('Already following this trader');
    }
    // No check: followerTelegramId !== leaderTelegramId
```

If User A follows their own address and places a trade, the copy-trading system detects the trade, then attempts to copy it back to User A. This creates a feedback loop where:

1. User A places a $10 trade.
2. The copy-trading system sees the trade and enqueues a copy trade for User A (following themselves).
3. The copy trade executes, which is detected as another trade by User A.
4. Step 2 repeats indefinitely.

With a `multiplier > 1` in proportional mode, this creates exponential trade amplification. With `percentage` mode allowing up to 1000% (`copy-trading.ts:170-172`), a single $10 trade could cascade into thousands of dollars of unintended positions.

Recommendations

1. Add a check in `follow()` : reject if `followerTelegramId` is equal to `leaderTelegramId` .
2. Add a circular follow detection: if A follows B, prevent B from following A (and deeper chains: A → B → C → A).
3. In the copy-trade execution worker, add a guard to skip trades where the follower and leader are the same user.

[L-38] ReferralService.mapRowToUser omits `isBanned` field

Description

The `ReferralService` in `packages/database/src/services/referrals.ts:153-180` has its own `mapRowToUser()` method that is missing several security-critical fields compared to the canonical `mapRowToUser()` in `users.ts:414-448` :

```
// referrals.ts:153-180 — MISSING FIELDS
private mapRowToUser(row: typeof users.$inferSelect): User {
    return {
        id: row.id,
        telegramId: row.telegramId,
        // ... basic fields ...
        tradingMode: (row.tradingMode as '...') ?? 'standard',
        tradingThreshold: row.tradingThreshold ?? '100',
        quickbuyPreset1: row.quickbuyPreset1 ?? '10',
        quickbuyPreset2: row.quickbuyPreset2 ?? '25',
        quickbuyPreset3: row.quickbuyPreset3 ?? '50',
        // MISSING: isBanned
        // MISSING: tradingFeePercent
        // MISSING: experimental
        // MISSING: americanOdds
        // MISSING: totpSecret
        // MISSING: totpEnabled
    };
}
```

Compare with `users.ts:414-448` which correctly maps all fields including `isBanned`, `tradingFeePercent`, `totpSecret`, `totpEnabled`, `experimental`, and `americanOdds`.

When `ReferralService` methods (`getDirectReferrals`, `getReferralChain`) return `User` objects, the `isBanned` property is `undefined`. In JavaScript/TypeScript, `if (user.isBanned)` evaluates `undefined` as falsy — meaning **banned users appear as not banned** when their `User` object comes from the referral service.

This also means:

- `tradingFeePercent` is `undefined`, which could cause fee calculation failures or zero-fee trades.
- `totpEnabled` is `undefined` (falsy), so 2FA checks would be skipped for referral-sourced users.

Recommendations

1. Remove the duplicate `mapRowToUser()` from `referrals.ts` and import the canonical mapper from `users.ts`, or refactor to a shared utility.
2. Alternatively, add all missing fields to `referrals.ts:mapRowToUser()` to match `users.ts:mapRowToUser()`.
3. Add a TypeScript strict type check that enforces all `User` interface fields are present in the return object (use `satisfies User` or ensure no `Partial`).

[L-39] Commission permanently lost if Redis enqueue fail

Description

The `referral_withdraw` handler at `apps/bot-v2/src/index.ts` executes these three steps in order:

```
// Step 1 — read the balance
const pendingCommission = parseFloat(user.pendingCommission || '0');

// Step 2 — zero it out immediately, before the job is confirmed
await userService.resetPendingCommission(user.id);

// Step 3 — enqueue the withdrawal job
await referralWithdrawQueue.add('referral-withdraw', {
    amount: pendingCommission,
    ...
});
```

`resetPendingCommission` sets `pending_commission = '0'` in the DB with no rollback path. If `referralWithdrawQueue.add()` at step 3 throws due to Redis being unavailable, connection timeout, or queue limit reached, then the commission balance is already 0 in the DB. And no worker job was ever created. The USDC is never transferred and the balance is never restored. The user has lost their commission permanently.

The worker's failure-recovery path at `userService.addPendingCommission(userId, amount)` at `referral-withdraw.worker.ts` only runs when a job exists and the on-chain transfer fails. It has no reach into the case where the job was never created.

Recommendations

- Enqueue the job first, then zero out `pendingCommission` only after the enqueue is confirmed:
- Add this method to `UserService` in `packages/database/src/services/users.ts`

```
async consumePendingCommission(userId: string): Promise<number> {
    const result = await this.db.execute(sql`
        UPDATE users
        SET pending_commission = '0', updated_at = NOW()
        WHERE id = ${userId}
        RETURNING pending_commission
    `);
    return parseFloat((result as any)[0]?.pending_commission || '0');
}
```

And update the handler:

```
// Step 1: atomically zero the balance and capture what was there
const amount = await userService.consumePendingCommission(user.id);

// Step 2: enqueue with the exact amount that was zeroed
try {
    await referralWithdrawQueue.add('referral-withdraw', { amount, ... });
} catch (err) {
    // Step 3: restore if enqueue fails — commission is not lost
    await userService.addPendingCommission(user.id, amount);
    throw err;
}
```

[L-40] checkDailyLimit() type mismatch bypasses daily limit

Description

In `packages/database/src/services/copy-trading.ts`, daily limit is checked by `checkDailyLimit()`, and its first parameter is `followId: string`; however, the `copyTradingFollows.id` column is serial (PostgreSQL integer, mapped to TypeScript number), resulting in a discrepancy. Every other method in `CopyTradingService` that accepts a `followId` types it as `number`.

The function internally calls `this.getFollowById(followId)` with the string. `getFollowById` exposes overloaded functions:

```
// copy-trading.ts:1010-1011
async getFollowById(followId: number): Promise<CopyTradingFollow | null>;
async getFollowById(followerTelegramId: string, followId: number): Promise<CopyTradingFollow
| null>;
```

Neither overload accepts a single string. TypeScript should raise a compile error, but if the project uses transpileOnly or skipLibCheck, this silently passes. At runtime, the implementation resolves to the string-first branch and executes:

```
// copy-trading.ts:1022-1027
where: (table, { eq, and }) => and(
    eq(table.followerTelegramId, followIdOrTelegramId), //@audit treated as telegramId
    eq(table.id, followId!)                             //@audit followId is undefined here
)
```

`followId` is undefined. Drizzle generates SQL with id = NULL, which matches no row. `getFollowById` returns null, and `checkDailyLimit` propagates its error branch:

```
// copy-trading.ts:827-829
const follow = await this.getFollowById(followId);
if (!follow) {
    throw new DatabaseError('Follow not found');
}
```

This implies `checkDailyLimit` always throws, or if the caller catches and defaults to allowing the trade, daily limits are never enforced.

Vulnerable code

```
// copy-trading.ts:819
async checkDailyLimit(followId: string, tradeAmount: number): Promise<{
    allowed: boolean;
    remainingLimit: number | null;
    currentSpent: number;
}> {
    const follow = await this.getFollowById(followId); // ← string, no matching overload
    ...
    const currentSpent = await this.getDailySpent(follow.id); // never reached
    ...
}
```

Recommendations

Change the parameter type to number:

```
async checkDailyLimit(followId: number, tradeAmount: number): ...
```

[L-41] `claimPromo` accepts arbitrary amounts

Description

The `claimPromo()` method in `packages/database/src/services/kol-promo.ts:122-149` accepts an arbitrary `amount` parameter and stores it directly as `claimedAmount` without validating it against the KOL promo's `claimableAmount`:

```
// kol-promo.ts:122
async claimPromo(userPromoId: string, txHash: string, amount: string): Promise<UserPromo> {
    // ...
    const [updated] = await this.db.update(userPromos)
        .set({
            status: 'claimed',
            claimedAmount: amount,  // No validation against kolPromos.claimableAmount!
            claimTxHash: txHash,
            // ...
        })
        .where(eq(userPromos.id, userPromoId))
        .returning();
}
```

The `kolPromos` table defines `claimableAmount` (`schema.ts:466`) with a default of `'20'` USDC. However, `claimPromo()` never looks up the associated `kolPromo` to verify `amount <= claimableAmount`. The method blindly trusts whatever the caller passes.

Recommendations

Look up the associated `kolPromo.claimableAmount` inside `claimPromo()` and verify `amount` is equal to `claimableAmount`.